

CVR COLLEGE OF ENGINEERING

(Autonomous)

SOFTWARE ENGINEERING & DEVOPS LAB

Laboratory Project Report

DevOps-Based Case Study with Automated CI/CD Pipeline
Case Study: Smart Parking Slot Booking System

Field	Details
Student Name	Mohammed Abdul Afnan
Roll Number	24B81A6666
Section / Branch	CSM - B
Faculty / Guide	Mr. U. Naresh
Subject	Software Engineering & DevOps Lab
Academic Year	2025 - 2026
GitHub	https://github.com/md-abdul-afnan/SmartParking-Devops
Submission Date	04 May 2026

Chapter 1: Introduction

The Smart Parking Slot Booking System is a Java-based console application that demonstrates core software engineering concepts integrated with an end-to-end DevOps workflow. The system manages parking slots, user bookings, cancellation flow, and availability status.

1.1 Objectives

- Develop Smart Parking Slot Booking System with booking and cancellation operations.
- Apply object-oriented design using model, service, and entry-point classes.
- Implement automated JUnit test cases to validate service layer behaviour.
- Set up CI/CD pipeline configuration using Jenkins stages.
- Containerise build artifact using Docker image and runtime container.
- Use Ansible playbook for automated deployment operations.
- Maintain source code in GitHub repository with proper commit history.

1.2 Tools and Technologies

Tool	Version	Purpose
Java JDK	11	Application development
Maven	3.x	Build and dependency management
JUnit	4.13.2	Unit testing framework
Git	2.x	Version control
Jenkins	LTS	Continuous Integration pipeline
Docker	24.x	Containerisation
Ansible	2.x	Configuration and deployment automation
GitHub	Cloud	Remote repository hosting

Chapter 2: System Development

2.1 Project Structure

- pom.xml – Maven project build configuration
- Dockerfile – image creation instructions
- Jenkinsfile – CI/CD pipeline stages
- deploy.yml – Ansible deployment playbook
- src/main/java/com/smartparking/model/ParkingSlot.java – slot entity class
- src/main/java/com/smartparking/model/Booking.java – booking entity class
- src/main/java/com/smartparking/service/ParkingService.java – business logic
- src/main/java/com/smartparking/Main.java – main execution file
- src/test/java/com/smartparking/service/ParkingServiceTest.java – JUnit tests

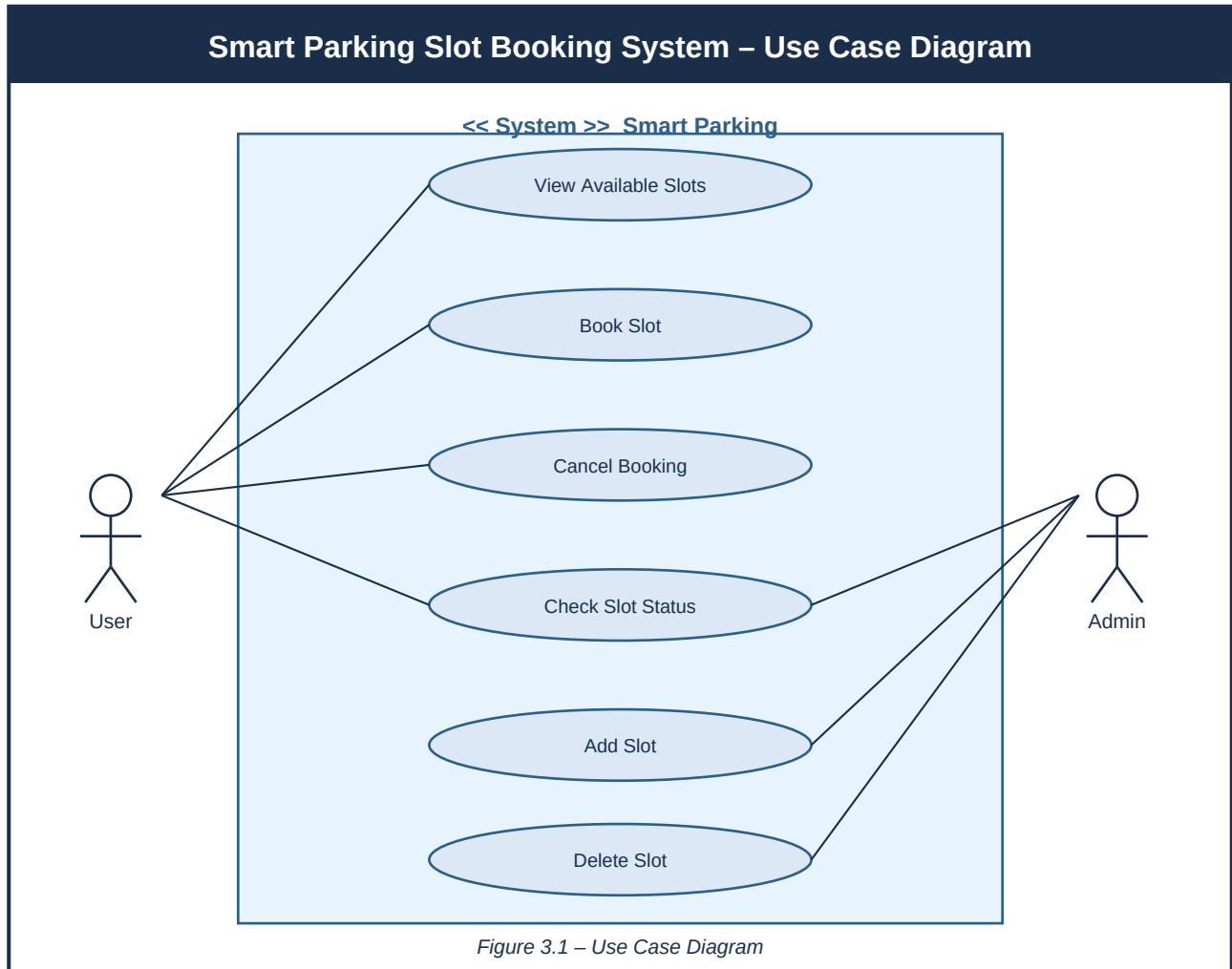
2.2 Class Descriptions

Class	Role	Key Methods
ParkingSlot	Model / Entity	getSlotId(), getSlotType(), isAvailable(), setAvailable()
Booking	Model / Entity	getBookingId(), getVehicleNumber(), getSlot()
ParkingService	Business Logic	addSlot(), bookSlot(), cancelBooking(), deleteSlot()
Main	Entry Point	main()
ParkingServiceTest	JUnit Test	12 test methods for service operations

Chapter 3: UML Diagrams

3.1 Use Case Diagram

The use case diagram illustrates interactions between two actors — User and Admin — and the Smart Parking Slot Booking System. The User can view available slots, book a slot, cancel a booking, and check slot status. The Admin can add or delete slots and also check slot status.



3.2 Class Diagram

The class diagram depicts five classes: ParkingSlot (entity), Booking (entity), ParkingService (business logic), Main (entry point), and ParkingServiceTest (JUnit). ParkingService aggregates both ParkingSlot and Booking objects and is used by Main and tested by ParkingServiceTest.

Smart Parking Slot Booking System – Class Diagram

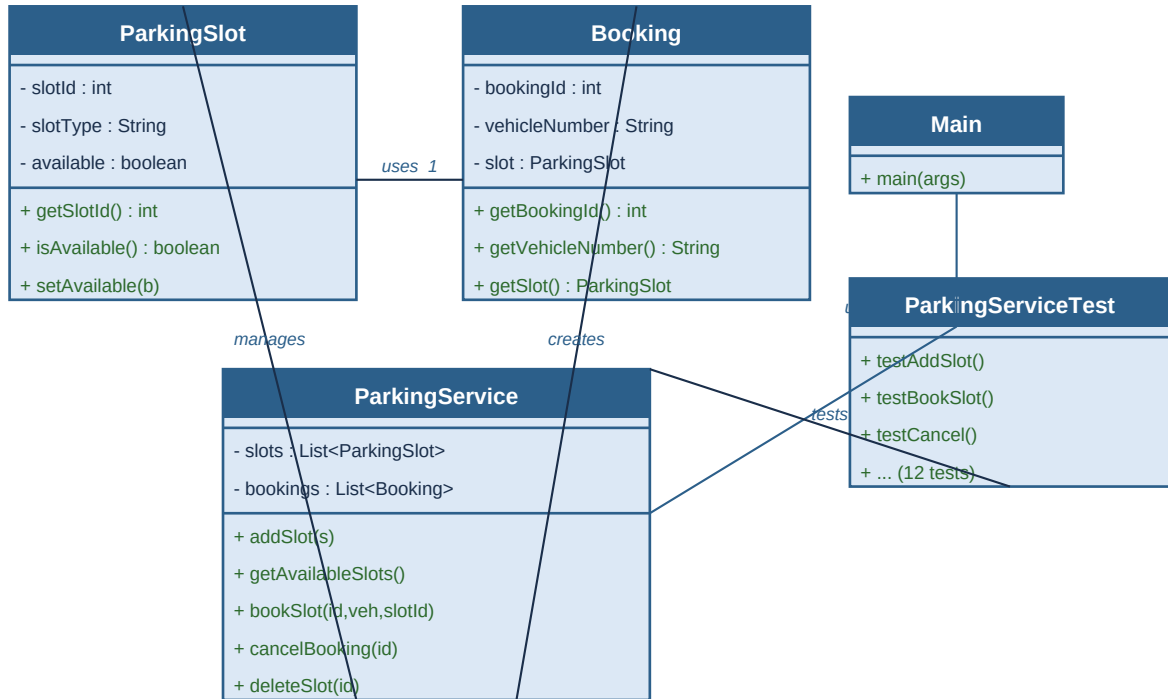
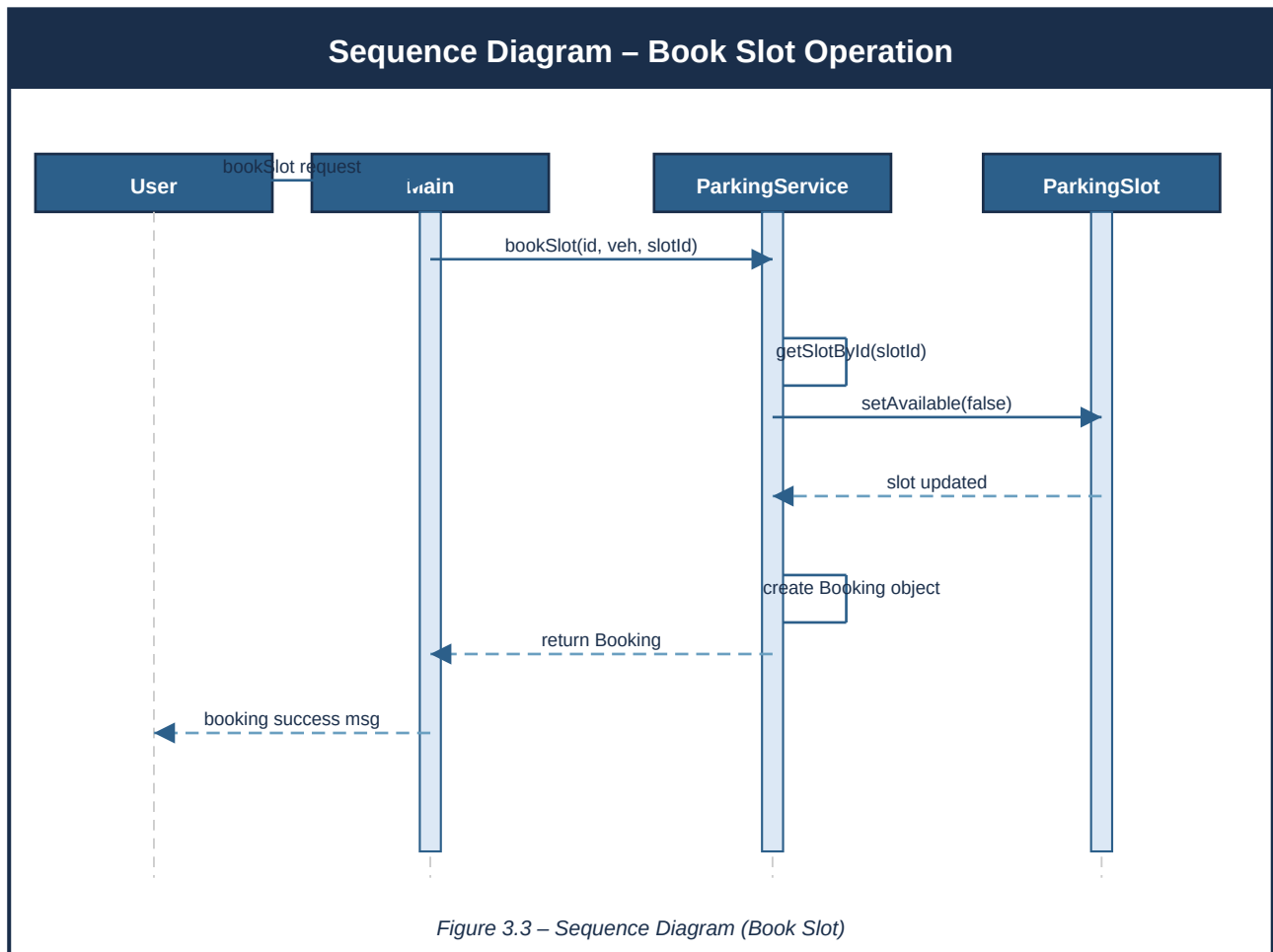


Figure 3.2 – Class Diagram

3.3 Sequence Diagram

The sequence diagram traces the message flow for the Book Slot operation. The User sends a request to Main, which delegates to ParkingService. ParkingService retrieves the slot, marks it unavailable, creates a Booking object, and returns a success response back to the User.



3.4 Deployment Diagram

The deployment diagram shows the full CI/CD pipeline: source code on the Developer PC is pushed to GitHub, which triggers the Jenkins server via a webhook. Jenkins builds, tests, packages, and creates a Docker image. Ansible automates the final deployment by pulling the image and running the container on the Docker Host.

Deployment Diagram – CI/CD & Container Pipeline

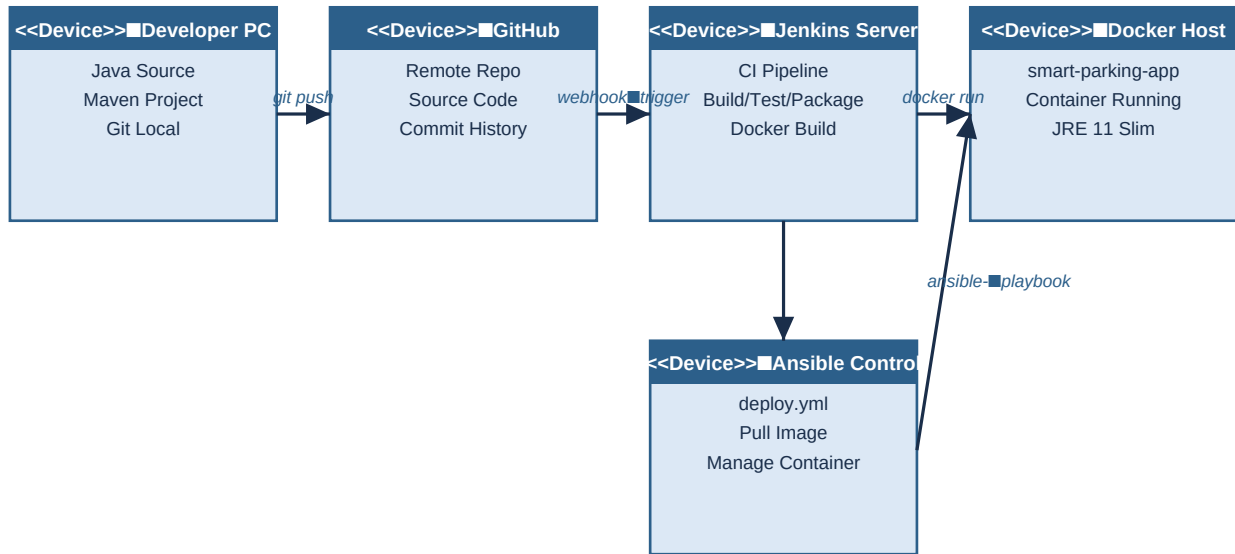


Figure 3.4 – Deployment Diagram

Chapter 4: Bug Identification & Test Cases

4.1 Identified Bugs

Bug ID	Description	Location	Status
BUG-01	Null slot accepted	ParkingService.addSlot()	FIXED
BUG-02	Duplicate slot IDs not blocked	ParkingService.addSlot()	FIXED
BUG-03	Unavailable slot booked	ParkingService.bookSlot()	FIXED
BUG-04	Vehicle number blank accepted	Booking constructor	FIXED
BUG-05	Deleting booked slot causes inconsistency	ParkingService.deleteSlot()	FIXED

4.2 Test Cases Summary

TC ID	Test Method	Expected Result	Status
TC-01	testAddSlot()	Slot count increments	PASS
TC-02	testAddDuplicateSlot()	IllegalArgumentException	PASS
TC-03	testGetSlotByIdFound()	Optional present	PASS
TC-04	testGetSlotByIdNotFound()	Optional empty	PASS
TC-05	testGetAvailableSlots()	Only available slots	PASS
TC-06	testUpdateSlotAvailabilitySuccess()	State updated	PASS
TC-07	testBookSlotSuccess()	Booking object returned	PASS
TC-08	testBookSlotAlreadyBooked()	IllegalStateException	PASS
TC-09	testCancelBookingSuccess()	Slot becomes available	PASS
TC-10	testDeleteSlotSuccess()	Slot removed	PASS

Chapter 5: Version Control – Git

Repository URL: <https://github.com/md-abdul-afnan/SmartParking-Devops>

5.1 Git Commands Used

Command	Purpose
git init	Initialize local repository
git add .	Stage all project files
git commit -m "msg"	Commit changes
git branch -M main	Set branch name to main
git remote add origin <url>	Attach GitHub remote repository
git push -u origin main	Push code to GitHub
git log --oneline	View commit history

5.2 Git Output (Sample)

```
C:\Users\...\SmartParking-DevOps> git init
Initialized empty Git repository
C:\Users\...\SmartParking-DevOps> git add .
C:\Users\...\SmartParking-DevOps> git commit -m "Initial commit"
C:\Users\...\SmartParking-DevOps> git push -u origin main
```

Chapter 6: Build Tool – Maven

Maven configuration is available in pom.xml. The project uses Java 11 and JUnit 4.13.2.

6.1 Maven Build Output

```
mvn clean compile test package
[INFO] Building smart-parking-slot-booking-system 1.0-SNAPSHOT
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Chapter 7: Continuous Integration – Jenkins

7.1 Pipeline Stages

Stage	Action
Clone	Checkout source code from GitHub
Build	Run mvn clean compile
Test	Run mvn test and publish reports
Package	Run mvn package -DskipTests
Docker Build	Build application image
Docker Run	Run smart-parking-app container

Chapter 8: Configuration Management – Ansible

Ansible playbook `deploy.yml` automates deployment steps: image pull, old container removal, and new container run.

```
ansible-playbook deploy.yml

PLAY [Deploy Smart Parking Slot Booking System]

TASK [Pull latest Docker image] ... ok
TASK [Remove old container if exists] ... changed
TASK [Start smart parking container] ... changed

PLAY RECAP: ok=3 changed=2 failed=0
```

Chapter 9: Containerisation – Docker

```
docker build -t smart-parking-slot-booking-system:latest .
docker run -d --name smart-parking-app smart-parking-slot-booking-system:latest
docker ps
```

The Dockerfile uses `openjdk:11-jre-slim` as base image and runs the built jar via `ENTRYPOINT`.

Chapter 10: Conclusion

This project successfully demonstrates integration of software development with DevOps practices through a Smart Parking Slot Booking System case study.

All lab requirements covered:

- ✓ System developed in Java with booking, cancel, and slot management.
- ✓ UML diagrams included: Use Case, Class, Sequence, Deployment.
- ✓ Bug identification and test case documentation completed.
- ✓ JUnit automated tests implemented for service logic.
- ✓ Git and GitHub repository setup completed.
- ✓ Maven build configuration completed.
- ✓ Jenkins CI pipeline file prepared.
- ✓ Ansible deployment playbook prepared.
- ✓ Docker containerisation setup completed.

Field	Details
Student Name	Mohammed Abdul Afnan
Roll Number	24B81A6666
Faculty	Mr. U. Naresh
GitHub	https://github.com/md-abdul-afnan/SmartParking-Devops
Submission Date	04 May 2026